Complete programs for the rank augmentation application

Human Compression project

Private draft; independent review pending

The first whole program supplies finite group computations for all three source examples. The second is a bounded diagnostic of the new common arguments and is not a premise of their quantified proofs. Neither is shortened to only its successful output. The separate eighteen-point correction has its own complete certificate, included in the dependency inventory rather than duplicated here.

Required source-example computations

```
# Complete three source examples, with actual parabolic and orbit computations.
SizeScreen([240,60]);;
Need:=function(x,m)if not x then Error(m);fi;end;;
Par:=function(xs,I)return Group(Concatenation([()],xs{I}));end;;
AllIP:=function(xs)local ss,i,j,n;ss:=Combinations([1..Length(xs)]);n:=0;
 for i in ss do for j in ss do
  Need(Intersection(Par(xs,i),Par(xs,j))=Par(xs,Intersection(i,j)),"old intersection property");n:=n+1;
 od;od;return n;end;;
cases:=[
 rec(n:=9,a:=(2,3)(6,9)(7,8),q:=(1,2)(3,4)(5,6),c:=(4,5)(6,7)(8,9),t:=(1,2),big:=[2..9],primitive:=true),
 rec(n:=10,a:=(1,2)(3,4)(9,10),q:=(2,3)(4,5)(6,7)(8,9),c:=(5,6)(7,8),t:=(2,3),big:=[3..10],primitive:=false),
 rec(n:=13,a:=(1,2)(3,4)(7,9)(8,10),q:=(2,3)(4,5)(6,7)(10,11)(12,13),c:=(5,6)(7,8)(9,10)(11,12),t:=(2,3),big:=[3..13],primitive:=true)
];;
rows:=[];;
for e in cases do
 old:=[e.a,e.q,e.c];;b:=e.t*e.q;;new:=[e.t,e.a,b,e.c];;
 Need(Size(Group(old))=Factorial(e.n),"original symmetric group");
 Need(ForAll(old,g->Order(g)=2) and e.a*e.c=e.c*e.a,"original string");
 count:=AllIP(old);;B:=Group([e.a,b]);;H:=Group([e.t,e.a,b]);;K:=Group([e.a,b,e.c]);;
 Need(ForAll(Orbits(B,[1..e.n]),o->Length(o)<=3),"middle orbit bound");
 Need(IsPrimitive(Action(K,e.big))=e.primitive,"specified primitive action");
 if e.n<>10 then Need(Size(Action(K,e.big))=Factorial(Length(e.big)),"full symmetric projection");fi;
 Need(Intersection(H,K)<>B,"new intersection failure");
 if e.n=9 then Need((2,3,4) in Intersection(H,K) and not (2,3,4) in B,"source first independent triple witness");fi;
 if e.n=10 then
  Need(Set(Orbit(K,[3,10],OnSets))=Set([[3,10],[4,9],[5,8],[6,7]]),"literal source block system");
  Need((1,2) in Intersection(H,K) and not (1,2) in B,"source pair-flip witness");
 fi;
 if e.n=13 then
  Need(ForAll([e.a,b,e.c],g->SignPerm(g)=1),"all three even marks");
  Need((3,4,5) in Intersection(H,K) and not (3,4,5) in B,"source third independent triple witness");
 fi;
 Add(rows,[e.n,Size(B),Size(H),Size(K),Size(Intersection(H,K)),List([e.a,b,e.c],SignPerm),count]);
od;
Need(rows=[[9,12,288,40320,72,[-1,1,-1],64],[10,12,1440,768,24,[-1,-1,1],64],[13,12,1440,39916800,72,[1,1,1],64]],"complete fixed source-example
table");
Print("SOURCE_EXAMPLES ",rows,"\n");
Print("PASS three source examples; old_IP_pairs=192; all output defects witnessed\n");
QUIT_GAP(0);
```

Bounded common-theorem diagnostics

```
# Exhaustive finite diagnostics within the explicitly bounded template space.
# These samples are not premises of any quantified written theorem.
SizeScreen([240,60]);;
Need:=function(x,m)if not x then Error(m);fi;end;;
Par:=function(xs,I)return Group(Concatenation([()],xs{I}));end;;
IsC:=function(xs)local ss,i,j;ss:=Combinations([1..Length(xs)]);
 for i in ss do for j in ss do
  if Intersection(Par(xs,i),Par(xs,j))<>Par(xs,Intersection(i,j)) then return false;fi;
 od;od;return true;end;;
TailList:=function(m,n)local tails,k,nxt,s,a,b,u,z;
 tails:=[[[],[],m]];
 for k in [1..n-m] do
  nxt:=[];
  for s in tails do a:=s[1];b:=s[2];u:=s[3];
   if u=n then Add(nxt,s);else
    Add(nxt,[a,b,u+1]);
    if u+2<=n then z:=(u+1,u+2);
     Add(nxt,[Concatenation(a,[z]),b,u+2]);
     Add(nxt,[a,Concatenation(b,[z]),u+2]);
     Add(nxt,[Concatenation(a,[z]),Concatenation(b,[z]),u+2]);
```

```
    fi;
    if u+3<=n then Add(nxt,[Concatenation(a,[(u+1,u+2)]),Concatenation(b,[(u+2,u+3)]),u+3]);fi;
   fi;
  od;tails:=Set(nxt);
 od;return Filtered(tails,s->s[3]=n);
end;;
templates:=[rec(m:=4,a:=(2,3),b:=(3,4),t:=(1,2),ends:=[1,2],u:=2),
 rec(m:=5,a:=(1,2)(3,4),b:=(4,5),t:=(2,3),ends:=[2,3],u:=3),
 rec(m:=6,a:=(2,3)(4,5),b:=(1,2)(5,6),t:=(3,4),ends:=[3,4],u:=4)];;
counts:=List([1..3],i->[0,0,0,0]);;R:=[];;
for d in [1..3] do e:=templates[d];
 for n in [Maximum(5,e.m)..8] do
  for tail in TailList(e.m,n) do
   a:=e.a*Product(tail[1],());;b:=e.b*Product(tail[2],());;t:=e.t;;
   central:=Intersection(Centralizer(SymmetricGroup(n),a),Stabilizer(SymmetricGroup(n),e.ends,OnTuples));;
   for c in Filtered(Elements(central),x->Order(x)=2) do
    counts[d][1]:=counts[d][1]+1;
    if not IsTransitive(Group([t,a,b,c]),[1..n]) then continue;fi;
    counts[d][2]:=counts[d][2]+1;
    Need(Length(Set([t,a,b,c]))=4,"distinct output involutions");
    B:=Group([a,b]);;H:=Group([t,a,b]);;K:=Group([a,b,c]);;
    Need(IsC([t,a,b]) and IsC([a,b,c]),"both actual C ends");
    full:=IsC([t,a,b,c]);;
    if e.m<6 then
     aout:=PermList(List([1..n],i->i));;bout:=aout;;
     aa:=List([1..n],i->i);;bb:=ShallowCopy(aa);;
     for i in [e.m+1..n] do aa[i]:=i^a;bb[i]:=i^b;od;
     Bout:=Group([PermList(aa),PermList(bb)]);;
     Need(H=Group(Concatenation(GeneratorsOfGroup(SymmetricGroup([1..e.m])),GeneratorsOfGroup(Bout))),"actual supported product");
     R:=Orbit(K,e.u);;triples:=Filtered(Orbits(B,[1..n]),o->Length(o)=3 and o[1] in R);;
     blockhyp:=Length(triples)=1 or not IsPrimitive(Action(K,R));;
     sufficient:=blockhyp and (e.m=4 or ForAll([a,b,c],x->SignPerm(x)=1));;
     if sufficient then Need(full,"four/five point positive theorem");counts[d][3]:=counts[d][3]+1;fi;
    else
     N:=Group([(3,4),(2,5),(1,6)]);;
     Need(Size(N)=8 and H=Group(Concatenation(GeneratorsOfGroup(N),[a,b])) and Size(Intersection(N,B))=1,"matching semidirect product");
     Need(full=(Size(Intersection(N,K))=1),"seven-test equivalence");
     if not 4 in Orbit(K,3) then Need(full,"separated-triple sufficient condition");counts[d][3]:=counts[d][3]+1;fi;
    fi;
    if full then counts[d][4]:=counts[d][4]+1;fi;
   od;
  od;
 od;
od;
Print("COUNTS [scanned,transitive,sufficient,C] ",counts,"\n");
Print("PASS bounded common-theorem diagnostics\n");
QUIT_GAP(0);
```